

Practical 1 (References)



What IDE are we using?

- Android Studio
 - It is officially supported by Google Android
 - Latest libraries for Android will be available here first
 - Students have been introduced to programming in Java
 - Java programming is object-oriented
 - And Java is useful not just for programming mobile applications for Android platform
 - Java is still very popular for programming enterprise software

Using Drag and Drop tools

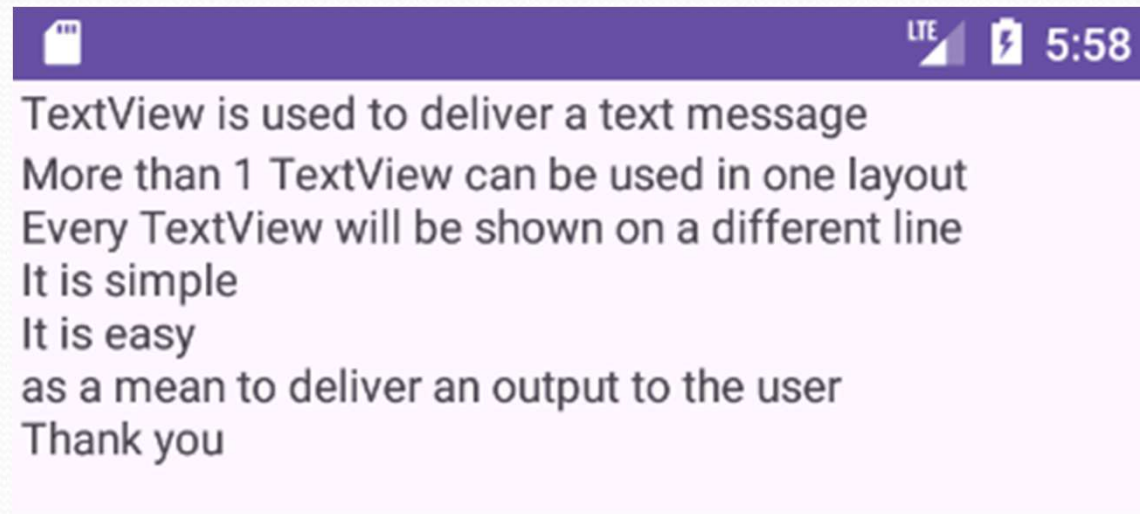
- You can easily drag and drop to create layouts
- You can create new layouts under res->layout
- You can preview how the layout looked like
- You can also check the XML source for the layout
- Please be brave to explore around and have fun!

Coding UI in Java

- If you need to create interfaces in realtime, you can write the codes in Java
- Check the following notes on examples of popular View components and their coding example in Java

TextView

- A standard read-only text label. It supports multiline display, string formatting, and automatic word wrapping.



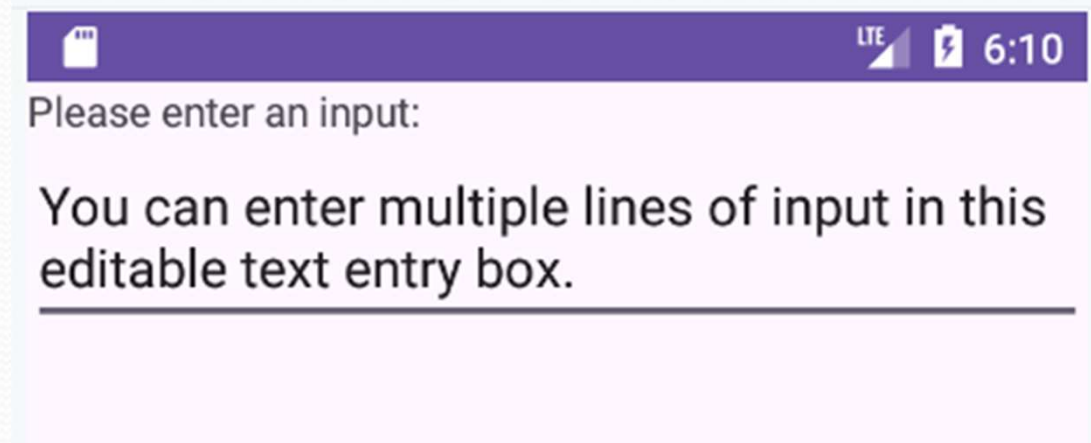
TextView - example

```
TextView text1 = new TextView(this);  
TextView text2 = new TextView(this);  
text1.setText("TextView is used to deliver a text  
message");  
text2.setText("More than 1 TextView can be used in  
one layout");
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(text1);  
ll.addView(text2);  
setContentView(ll);
```

EditText

- An editable text entry box.
- It accepts multiline entry, wordwrapping, and hint text.

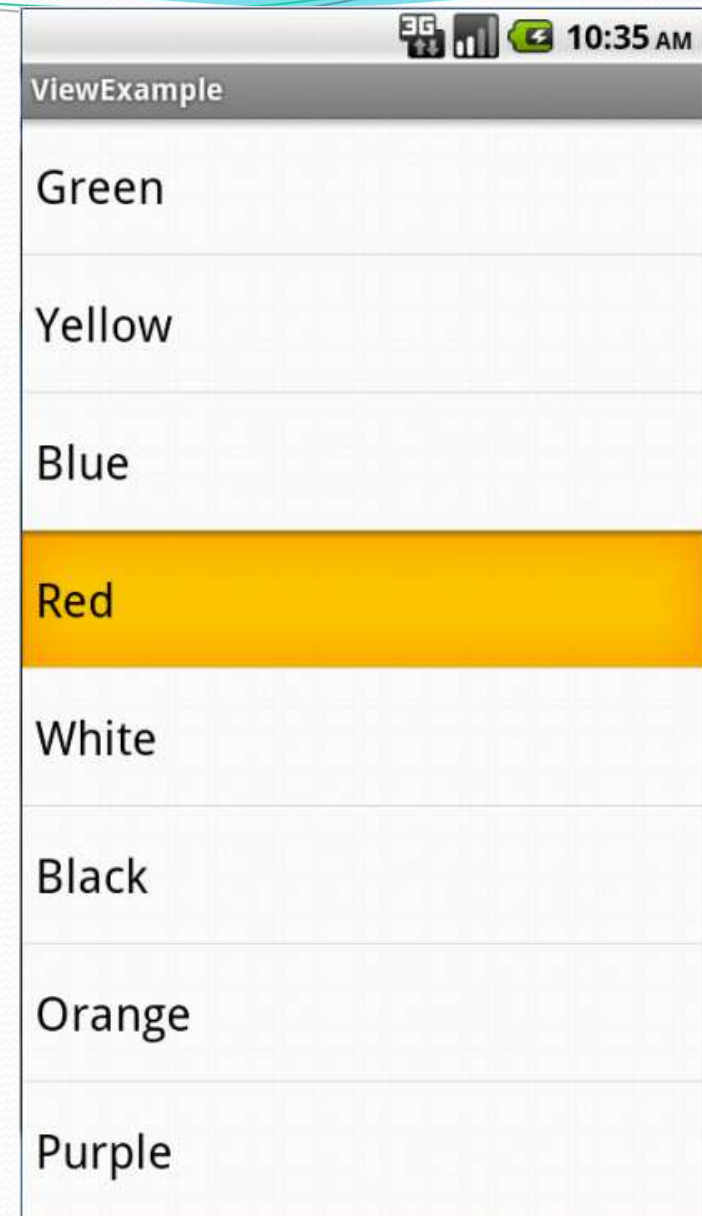


EditText - example

```
TextView text = new TextView(this);  
text.setText("Please enter an input:");  
EditText edit = new EditText(this);  
  
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(text);  
ll.addView(edit);  
setContentView(ll);
```

ListView

- A View Group that creates and manages a vertical list of Views, displaying them as rows within the list.
- The simplest List View displays the **toString** value of each object in an array, using a Text View for each item.
- Created using **ListActivity**

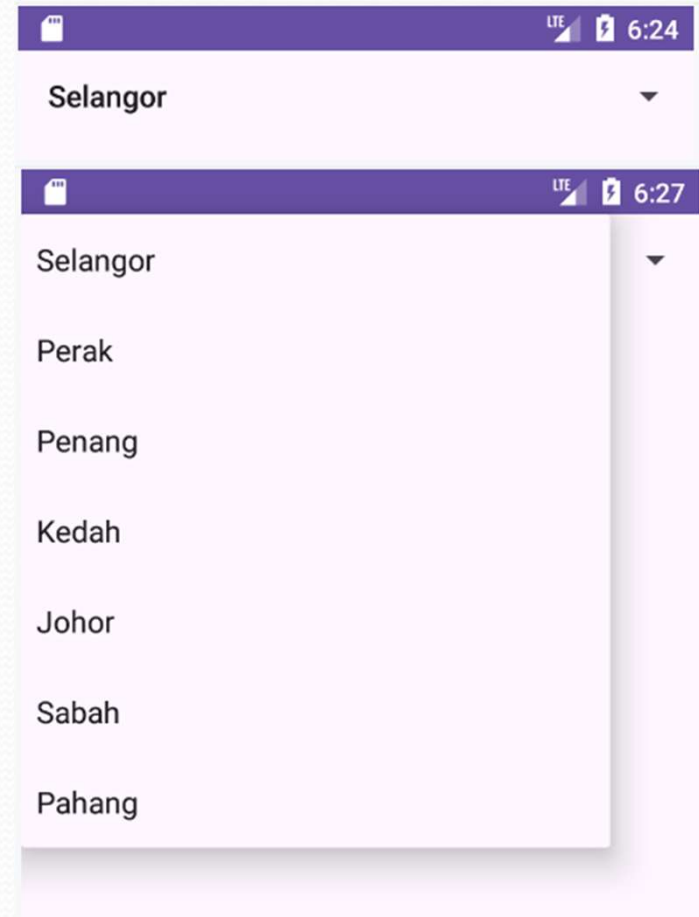


ListView - example

```
public class ColorActivity extends ListActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
  
        String[] colors = {"Green", "Yellow", "Blue", "Red", "White",  
                            "Black", "Orange", "Purple", "Pink", "Grey"};  
  
        ArrayAdapter<String> adapter = new ArrayAdapter<String>(  
            this, android.R.layout.simple_list_item_1, colors);  
  
        setListAdapter(adapter);  
    }  
}
```

Spinner

- A composite control that displays a Text View and an associated List View that lets you select an item from a list to display in the textbox.
- The list of items will only show up when the user clicks on the Spinner, thus taking up less space than typical List View.

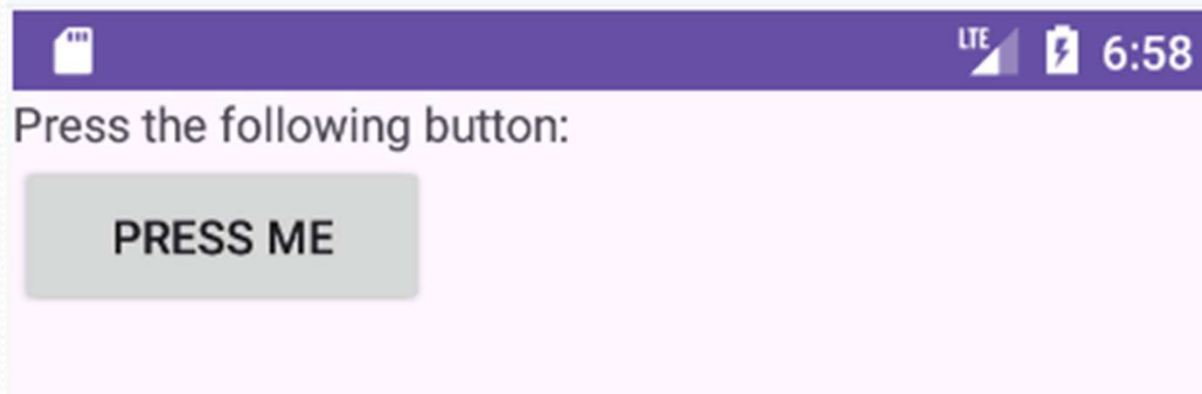


Spinner - example

```
String[] items =  
    {"Selangor", "Perak", "Penang", "Kedah", "Johor", "Sabah", "Pahang"};  
  
ArrayAdapter<String> adapter = new  
    ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, i  
        tems);  
adapter.setDropDownViewResource(android.R.layout.simple_spinner_d  
    rotdown_item);  
Spinner spin = new Spinner(this);  
  
spin.setAdapter(adapter);  
  
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(spin);  
setContentView(ll);
```

Button

- A standard push-button.
- Allows user to trigger for an action, as a form of confirmation from the user, typically to mark the end of user input.



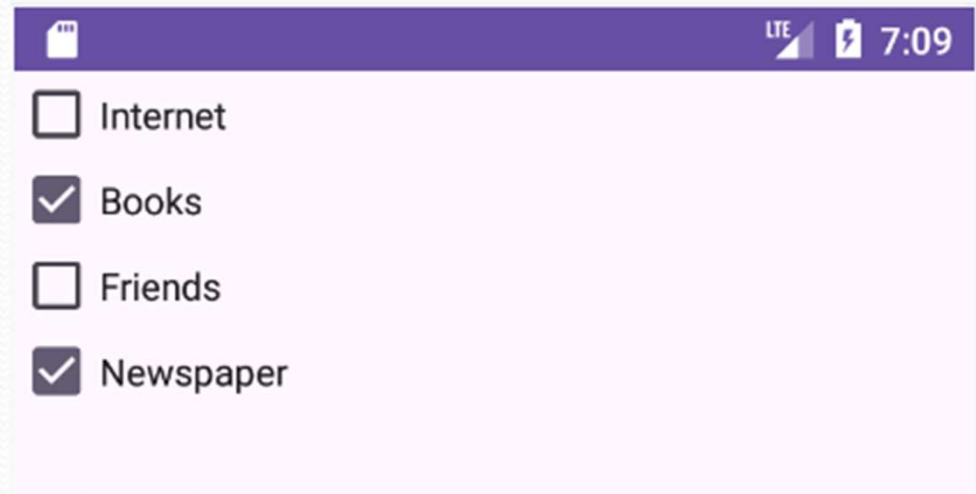
Button - example

```
Button button = new Button(this);
button.setText("Press Me");
ViewGroup.LayoutParams lp = new
    ViewGroup.LayoutParams(250,
        ViewGroup.LayoutParams.WRAP_CONTENT);
button.setLayoutParams(lp);
TextView text = new TextView(this);
text.setText("Press the following button:");
```

```
LinearLayout ll = new LinearLayout(this);
ll.setOrientation(LinearLayout.VERTICAL);
ll.addView(text);
ll.addView(button);
setContentView(ll);
```

CheckBox

- A two-state button represented by a checked or unchecked box
- The users get to choose none or more than one choice in such scenario



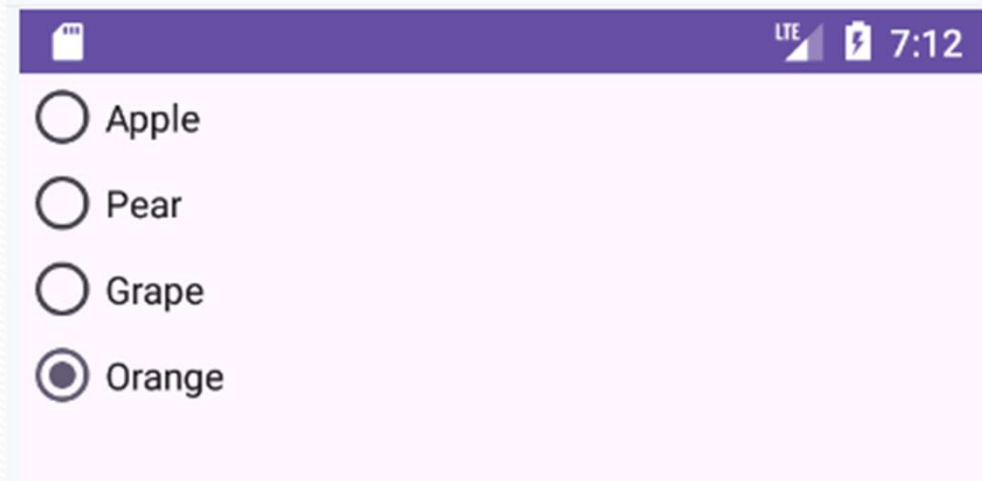
CheckBox - example

```
CheckBox chk1 = new CheckBox(this);  
CheckBox chk2 = new CheckBox(this);  
CheckBox chk3 = new CheckBox(this);  
CheckBox chk4 = new CheckBox(this);  
chk1.setText("Internet");  
chk2.setText("Books");  
chk3.setText("Friends");  
chk4.setText("Newspaper");
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(chk1);  
ll.addView(chk2);  
ll.addView(chk3);  
ll.addView(chk4);  
setContentView(ll);
```

RadioButton

- A two-state grouped button. A group of these presents the user with a number of binary options where only one can be enabled at a time.



RadioButton - example

```
RadioButton rb1 = new RadioButton(this);  
RadioButton rb2 = new RadioButton(this);  
RadioButton rb3 = new RadioButton(this);  
RadioButton rb4 = new RadioButton(this);  
rb1.setText("Apple");  
rb2.setText("Pear");  
rb3.setText("Grape");  
rb4.setText("Orange");
```

```
RadioGroup rg = new RadioGroup(this);  
rg.addView(rb1);  
rg.addView(rb2);  
rg.addView(rb3);  
rg.addView(rb4);
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(rg);  
setContentView(ll);
```

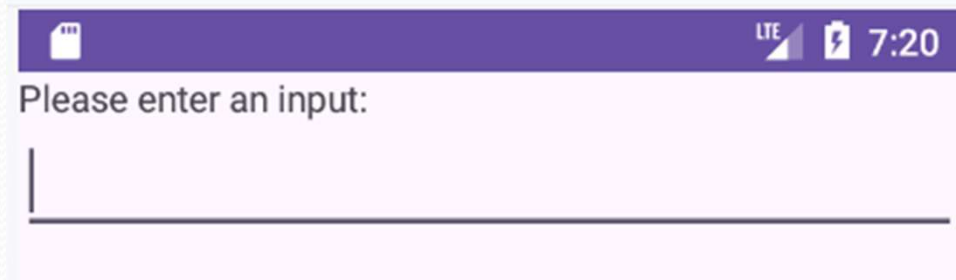
LinearLayout

- Aligns each child View in either a vertical or horizontal line
- A vertical layout has a column of Views, while a horizontal layout has a row of Views
- The Linear Layout manager enables you to specify a “weight” for each child View that controls the relative size of each within the available space

LinearLayout - example

```
TextView text = new TextView(this);  
text.setText("Please enter an input:");  
EditText edit = new EditText(this);
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(text);  
ll.addView(edit);  
setContentView(ll);
```



RelativeLayout

- The most flexible of the native layouts, the RelativeLayout lets you define the positions of each child View relative to the others and to the screen boundaries

RelativeLayout - example

```
RelativeLayout rl = new RelativeLayout(this);
RelativeLayout.LayoutParams params = new
    RelativeLayout.LayoutParams(RelativeLayout.LayoutParams.
        WRAP_CONTENT,RelativeLayout.LayoutParams.WRAP_CONTENT);
rl.setLayoutParams(params);
```

```
TextView text = new TextView(this);
text.setId(1);
text.setText("Please enter an input:");
params.addRule(RelativeLayout.ALIGN_PARENT_TOP,1);
params.addRule(RelativeLayout.ALIGN_PARENT_LEFT,1);
text.setLayoutParams(params);
```

RelativeLayout - example

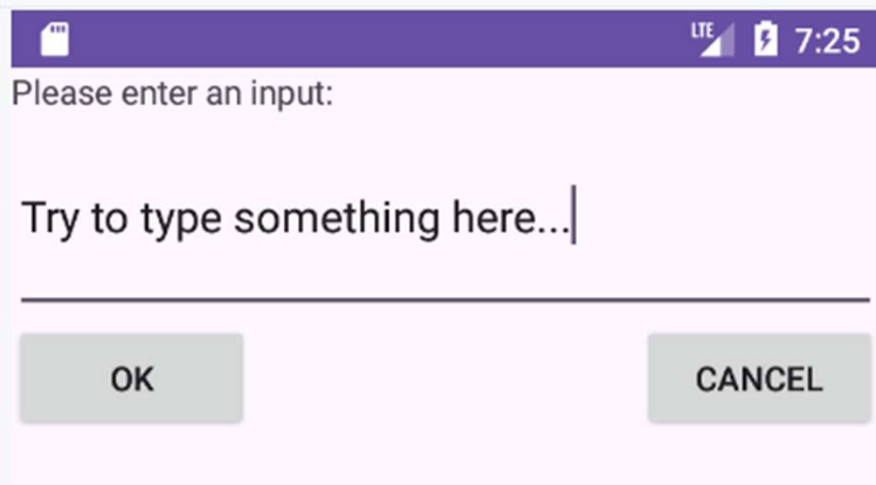
```
EditText edit = new EditText(this);
RelativeLayout.LayoutParams params2 = new
    RelativeLayout.LayoutParams(RelativeLayout.LayoutParams.MATCH_PARENT, 170);
edit.setId(2);
params2.addRule(RelativeLayout.ALIGN_LEFT, text.getId());
params2.addRule(RelativeLayout.BELOW, text.getId());
params2.addRule(RelativeLayout.CENTER_HORIZONTAL, 1);
edit.setLayoutParams(params2);

Button but1 = new Button(this);
RelativeLayout.LayoutParams params3 = new RelativeLayout.LayoutParams(200,
    RelativeLayout.LayoutParams.WRAP_CONTENT);
but1.setId(3);
but1.setText("OK");
params3.addRule(RelativeLayout.ALIGN_LEFT, edit.getId());
params3.addRule(RelativeLayout.BELOW, edit.getId());
but1.setLayoutParams(params3);
```

RelativeLayout - example

```
Button but2 = new Button(this);
RelativeLayout.LayoutParams params4 = new
    RelativeLayout.LayoutParams(200,
        RelativeLayout.LayoutParams.WRAP_CONTENT);
but2.setId(4);
but2.setText("Cancel");
params4.addRule(RelativeLayout.ALIGN_RIGHT, edit.getId());
params4.addRule(RelativeLayout.BELOW, edit.getId());
but2.setLayoutParams(params4);

rl.addView(text);
rl.addView(edit);
rl.addView(but1);
rl.addView(but2);
setContentView(rl);
```



TableLayout

- Allows you to lay out Views using a grid of rows and columns
- Tables can span multiple rows and columns, and columns can be set to shrink or grow

TableLayout - example

```
TableLayout tl = new TableLayout(this);
tl.setLayoutParams(new
    TableRow.LayoutParams(TableRow.LayoutParams.MATCH_PARENT
        , TableRow.LayoutParams.MATCH_PARENT));
tl.setStretchAllColumns(true);

TableRow row1 = new TableRow(this);
TextView text1 = new TextView(this);
text1.setText("Message 1");
TextView text2 = new TextView(this);
text2.setText("Message 2");
row1.addView(text1);
row1.addView(text2);
tl.addView(row1);
```

TableLayout - example

```
TableRow row2 = new TableRow(this);  
TextView text3 = new TextView(this);  
text3.setText("Message 3");  
TextView text4 = new TextView(this);  
text4.setText("Message 4");  
row2.addView(text3);  
row2.addView(text4);  
t1.addView(row2);  
  
setContentView(t1);
```

